Divide-and-Conquer Algorithms

Consider a problem P associated with a set S. Suppose A is an algorithm which partitions S into smaller sets such that the solution of the problem P for S is reduced to the solution of P for one or more of the smaller sets. Then A is called a divide-and-conquer algorithm.

Two examples of divide-and-conquer algorithms, previously treated, are the quicksort algorithm in Sec. 6.6 and the binary search algorithm in Sec. 4.7. Recall that the quicksort algorithm uses a reduction step to find the location of a single element and to reduce the problem of sorting the entire set to the problem of sorting smaller sets. The binary search algorithm divides the given sorted set into two halves so that the problem of searching for an item in the entire set is reduced to the problem of searching for the item in one of the two halves.

A divide-and-conquer algorithm A may be viewed as a recursive procedure. The reason for this is that the algorithm A may be viewed as calling itself when it is applied to the smaller sets. The base criteria for these algorithms are usually the one-element sets. For example, with a sorting algorithm, a one-element set is automatically sorted; and with a searching algorithm, a one-element set requires only a single comparison.

Ackermann Function

The Ackermann function is a function with two arguments each of which can be assigned any nonnegative integer: 0, 1, 2,.... This function is defined as follows:

Definition 6.3: (Ackermann Function)

(a) If $m = 0$, then $A(m, n) = n + 1$.
(b) If $m \neq 0$ but $n = 0$, then $A(m, n) = A(m - 1, 1)$.
(c) If $m \neq 0$ and $n \neq 0$, then $A(m, n) = A(m - 1, A(m, n - 1))$

Once more, we have a recursive definition, since the definition refers to itself in parts (b) and (c). Observe that $A(m, n)$ is explicitly given only when $m = 0$. The base criteria are the pairs

$$(0, 0), \quad (0, 1), \quad (0, 2), \quad (0, 3), \ldots, (0, n), \ldots$$

Although it is not obvious from the definition, the value of any $A(m, n)$ may eventually be expressed in terms of the value of the function on one or more of the base pairs.

The value of $A(1, 3)$ is calculated in Solved Problem 6.18. Even this simple case requires 15 steps. Generally speaking, the Ackermann function is too complex to evaluate on any but a trivial example. Its importance comes from its use in mathematical logic. The function is stated here mainly to give another example of a classical recursive function and to show that the recursion part of a definition may be complicated.

6.8 TOWERS OF HANOI

The preceding section gave examples of some recursive definitions and procedures. This section shows how recursion may be used as a tool in developing an algorithm to solve a particular problem. The problem we pick is known as the Towers of Hanoi problem.

Suppose three pegs, labeled A, B and C, are given, and suppose on peg A there are placed a finite number n of disks with decreasing size. This is pictured in Fig. 6.14 for the case $n = 6$. The object of the game is to move the disks from peg A to peg C using peg B as an auxiliary. The rules of the game are as follows:

(a) Only one disk may be moved at a time. Specifically, only the top disk on any peg may be moved to any other peg.

(b) At no time can a larger disk be placed on a smaller disk.

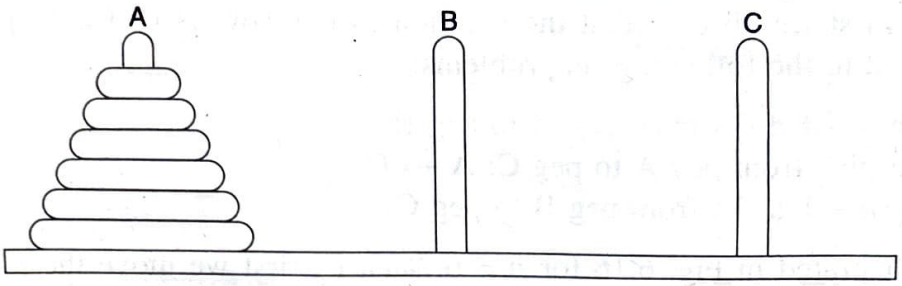


Fig. 6.14 *Initial Setup of Towers of Hanoi with n = 6*

Sometimes we will write X → Y to denote the instruction "Move top disk from peg X to peg Y," where X and Y may be any of the three pegs.

The solution to the Towers of Hanoi problem for $n = 3$ appears in Fig. 6.15. Observe that it consists of the following seven moves:

$n = 3$: Move top disk from peg A to peg C.
Move top disk from peg A to peg B.
Move top disk from peg C to peg B.
Move top disk from peg A to peg C.
Move top disk from peg B to peg A.
Move top disk from peg B to peg C.
Move top disk from peg A to peg C.

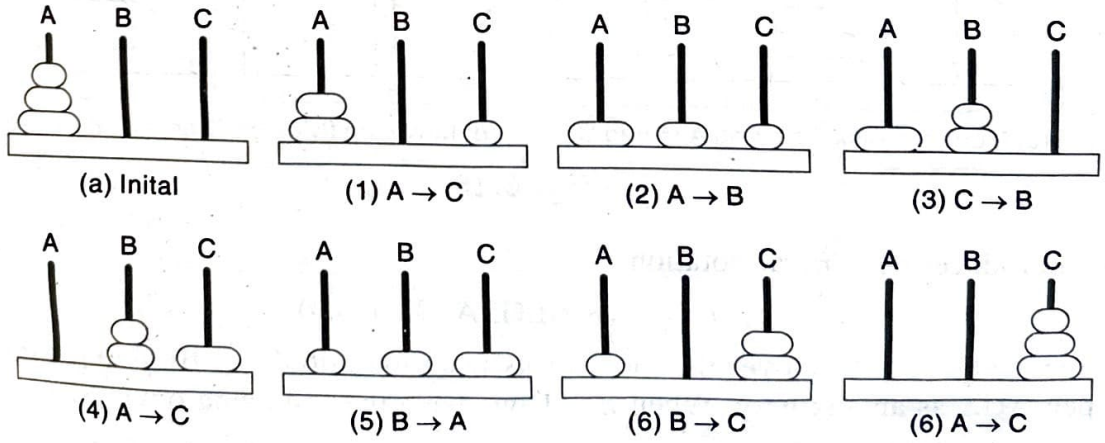


Fig. 6.15

In other words,

$$n = 3: \quad A \to C, \quad A \to B, \quad C \to B, \quad A \to C, \quad B \to A, \quad B \to C, \quad A \to C$$

For completeness, we also give the solution to the Towers of Hanoi problem for $n = 1$ and $n = 2$:

$$n = 1: \quad A \to C$$
$$n = 2: \quad A \to B, \quad A \to C, \quad B \to C$$

Note that $n = 1$ uses only one move and that $n = 2$ uses three moves.

Rather than finding a separate solution for each n, we use the technique of recursion to develop a general solution. First we observe that the solution to the Towers of Hanoi problem for $n > 1$ disks may be reduced to the following subproblems:

(1) Move the top $n - 1$ disks from peg A to peg B.
(2) Move the top disk from peg A to peg C: A → C.
(3) Move the top $n - 1$ disks from peg B to peg C.

This reduction is illustrated in Fig. 6.16 for $n = 6$. That is, first we move the top five disks from peg A to peg B, then we move the large disk from peg A to peg C, and then we move the top five disks from peg B to peg C.

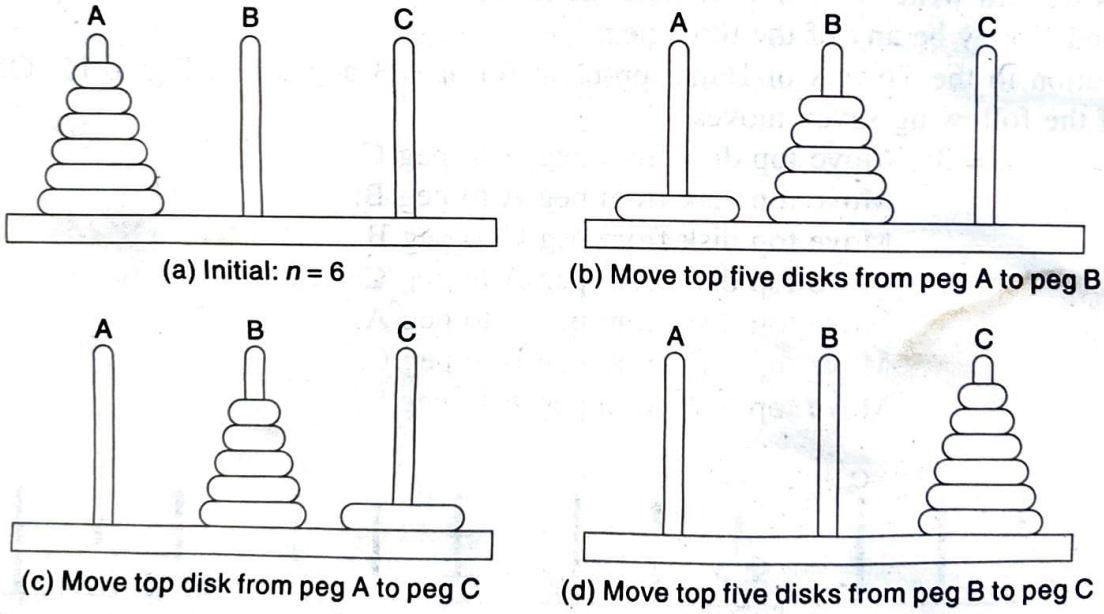


Fig. 6.16

Let us now introduce the general notation

TOWER(N, BEG, AUX, END)

to denote a procedure which moves the top n disks from the initial peg BEG to the final peg END using the peg AUX as an auxiliary. When $n = 1$, we have the following obvious solution:

TOWER(1, BEG, AUX, END) consists of the single instruction BEG → END

Furthermore, as discussed above, when $n > 1$, the solution may be reduced to the solution of the following three subproblems:

(1) TOWER(N – 1, BEG, END, AUX)
(2) TOWER(1, BEG, AUX, END) or BEG → END
(3) TOWER(N – 1, AUX, BEG, END)

Observe that each of these three subproblems may be solved directly or is essentially the same as the original problem using fewer disks. Accordingly, this reduction process does yield a recursive solution to the Towers of Hanoi problem.

Figure 6.17 contains a schematic diagram of the above recursive solution for

TOWER(4, A, B, C)

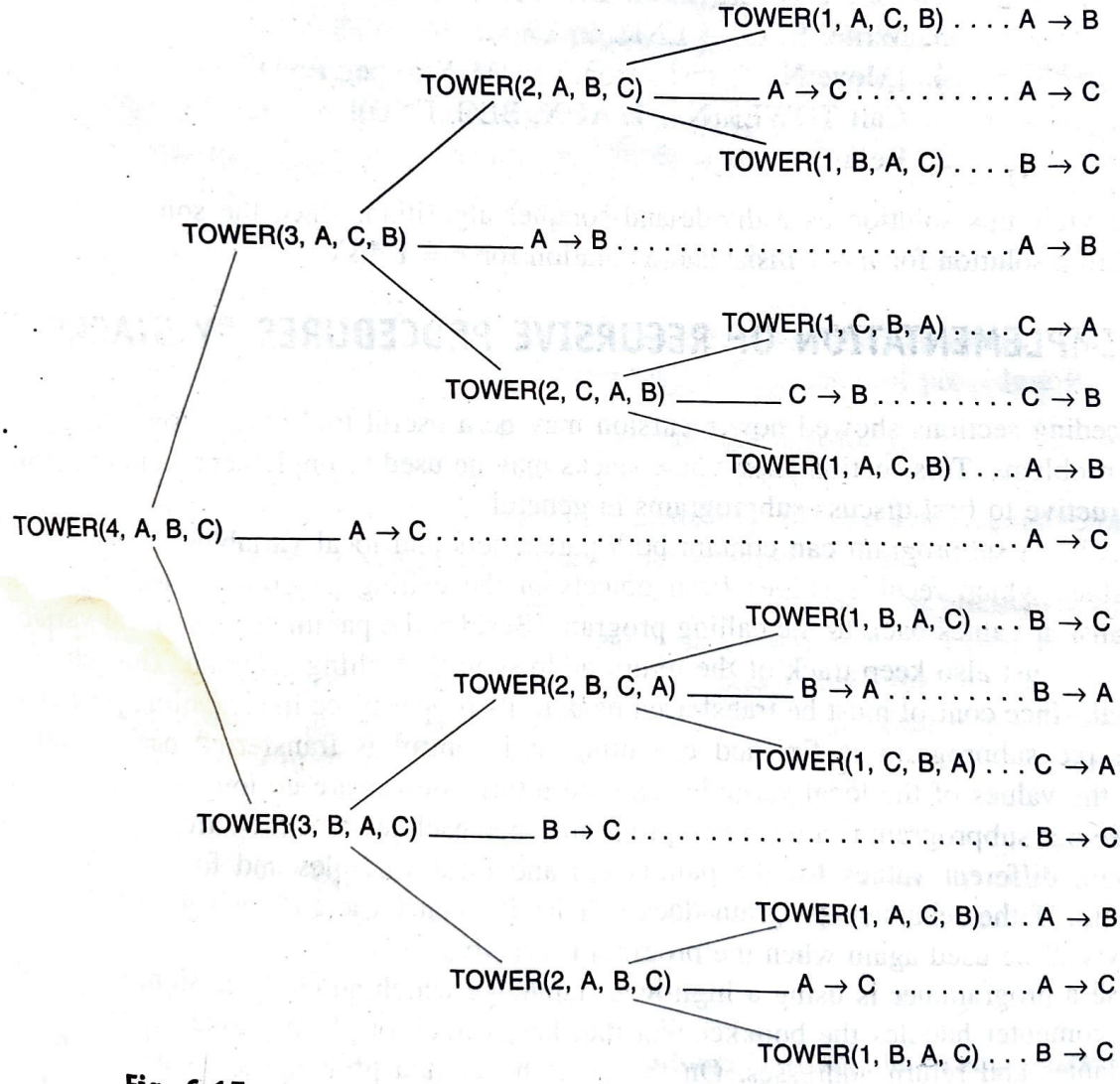


Fig. 6.17 *Recursive Solution to Towers of Hanoi Problem for n = 4*

Observe that the recursive solution for $n = 4$ disks consists of the following 15 moves:

$$\begin{array}{llllllll} A \to B & A \to C & B \to C & A \to B & C \to A & C \to B & A \to B & A \to C \\ B \to C & B \to A & C \to A & B \to C & A \to B & A \to C & B \to C & \end{array}$$

In general, this recursive solution requires $f(n) = 2^n - 1$ moves for n disks.

We summarize our investigation with the following formally written procedure.

Procedure 6.11: TOWER(N, BEG, AUX, END)

This procedure gives a recursive solution to the Towers of Hanoi problem for N disks.

1. If N = 1, then:
   (a) Write: BEG → END.
   (b) Return.
   [End of If structure.]
2. [Move N – 1 disks from peg BEG to peg AUX.]
   Call TOWER(N – 1, BEG, END, AUX).
3. Write: BEG → END.
4. [Move N – 1 disks from peg AUX to peg END.]
   Call TOWER(N – 1, AUX, BEG, END).
5. Return.

One can view this solution as a divide-and-conquer algorithm, since the solution for n disks is reduced to a solution for $n - 1$ disks and a solution for $n = 1$ disk.